

STFL: Utilizing a Semi-Supervised, Transfer-Learning, Federated-Learning Approach to Detect Phishing URL Attacks

Ido Sakazi*, Editia Grolman*, Yuval Elovici*, and Asaf Shabtai*

Department of Software and Information Systems Engineering, Ben-Gurion University of the Negev, Israel
{sakazi, edita}@post.bgu.ac.il, {elovici, shabtaia}@bgu.ac.il

Abstract—Phishing attacks are continually changing, so machine learning detection models must be continuously updated by collecting new URL data from users without compromising their privacy. Existing approaches for the detection of phishing URLs have unrealistic assumptions: (1) representative URL datasets are available in a centralized location, (2) users' URL entries are labeled, (3) users' unique behavioral patterns are ignored, and (4) users' data are identically and independently distributed (IID data). This paper presents a semi-supervised, transfer-learning (TL), federated-learning (FL) approach for detecting phishing URL attacks, a novel approach that does not hold the above assumptions. We train a bidirectional long short-term memory (Bi-LSTM) autoencoder network across multiple decentralized edge devices (using FL) containing unlabeled data samples without sharing them (the process is privacy-preserving). A centralized server collects the updated Bi-LSTM autoencoder networks from the users' devices and aggregates them into a global Bi-LSTM autoencoder network using the FedAVG algorithm. The server then performs TL in order to use the autoencoder that learns the patterns from the global Bi-LSTM autoencoder networks and induces a classification model. The method is evaluated using three benchmark datasets and compared to state-of-the-art URL phishing detection methods that utilize centralized learning (CL) and FL. Our experiments show that our proposed approach achieves higher results based on the F1 score compared to the state-of-the-art method.

Index Terms—federated learning, phishing detection, transfer learning, semi-supervised learning, centralized learning, anomaly detection

I. INTRODUCTION

Phishing is a well-known type of social engineering attack used to manipulate online users so they make poor security decisions or give away sensitive and personal information about themselves (e.g., ID numbers, usernames, passwords, email addresses, phone numbers, and credit card details) [4], [14], [19], [44]. According to the fourth-quarter of 2023 report of the Anti-Phishing Working Group (APWG) [2], there has been rapid growth in phishing attacks on websites, where the industry sectors most targeted by phishing attacks are social media, webmail sites, financial institutions, eCommerce/retail, logistics/shipping, payment, and telecom sites. The most common way for attackers to obtain sensitive information from online users is by sending a URL link to a malicious website (i.e., a fake website that seems legitimate which is used to manipulate users into revealing sensitive personal information). In

these kinds of attacks, the attacker usually imitate the popular webpages' identities by sending emails, short message service (SMS), or instant messenger along with a phishing URL [4], [44].

Since phishing URLs have plagued the cyber world for over two decades [8], many different approaches have been developed to determine whether a URL link is phishing or benign; these approaches include: (1) blacklist/whitelist filtering [23], [6], [44], (2) website content-based analysis [53], [34], [54], and (3) URL character-based inspection [3], [45], [46], [49], [42], [11], [12]. In this paper, we focus on the URL character-based approach, which is more common today [28]. Our proposed method addresses the following challenges: **Users' privacy.** Most existing methods for detecting phishing URLs assume that the training data are available in one place, i.e. centralized learning (CL); however, when the data are centralized, the end user's privacy can be violated [37]. Thus, there is a need to perform training across multiple decentralized edge devices that contain data samples, without sharing them. This kind of training approach can be performed using a federated learning (FL) framework, a framework used in our method. In FL, users train a global model jointly by sharing just the local models with the server. The users' data remains on their devices, and their data's privacy are preserved [27], [30], [37]. **Unlabeled data.** In real-world environments, there is no indication whether a URL accessed by the users is legitimate or not; hence, the user's URLs entries are unlabeled. Thus, we opted to use unsupervised learning techniques in order to learn the unlabeled URL data patterns. **Non-independent and non-identically distributed data.** In a real-world scenario, each user generates their dataset based on their behavior and device usage. Therefore, it can not be assumed that all users' data are independent and identically distributed (IID). Our method assumes heterogeneous behavior by considering each user's unique behavioral patterns. We induce a model according to each user's device in order to learn the representation of that user's data which is unlabeled. In addition, our training process on each user's device is based on unsupervised learning, and is thus unaffected by non-IID URL data.

We propose STFL, a semi-supervised, transfer learning (TL), FL approach for the detection of phishing URLs. The use of FL training enables our method to preserve end users'

Corresponding authors.

sensitive and private data, since their URL data are not shared with the server. In addition, the training process on the user's device is based on unsupervised learning. On the user's device, a bidirectional long short-term memory (Bi-LSTM) [15] autoencoder (AE) network is trained on unlabeled URL data, which are then sent to the server to be aggregated into a global Bi-LSTM autoencoder using the FedAVG algorithm, which is the most common optimization method in FL [35]. The server 1) conducts TL by removing the encoder from the updated global Bi-LSTM AE and 2) induces a classification model by adding it to the fully connected layers. Then, the server trains the classification model as supervised learning with a publicly available labeled dataset.

To the best of our knowledge, the research studies that combine FL to detect malicious URLs (including phishing URLs) were Khramtsova et al. [25] and Ongun et al. [39]. However, they assumed that each user's URLs were labeled (i.e., fully supervised learning). We compare our method with the publicly available method proposed by Khramtsova et al. To compare our method with previous study, we modified our method and tested it in both settings, supervised and semi-supervised. We show that our method achieves better results in both of these cases as well as when using centralized learning. The main contributions of the proposed STFL method are: (1) it is suitable for real-world settings; (2) it can handle users' heterogeneous data and non-IID data; (3) it is capable of learning the unique behavior patterns in each user's data;

II. BACKGROUND

A. Federated Learning (FL)

FL is a distributed DL approach in which training is performed across multiple decentralized edge devices that contain data samples, without sharing them [27], [35]. FL algorithms use a server with an initialized or pretrained global model that was collaboratively trained across different users. The users jointly train the global model by sharing only the updated local models, which are then aggregated and used to update the global model (in the server). Thus, the global model is constantly updated using the updated local models of the users, without sharing the users' data with the server, thereby maintaining users' privacy. FL has been applied in many applications, including: Mobile apps [20], [41], [35], [33], IoT devices [55], [38], Cybersecurity [10], [40], electric vehicles [32], healthcare [9], [50], [31], and recommendation systems (RSs) [36]. One of the most common optimization methods for FL is the FedAvg algorithm [35]. The FedAvg algorithm is used on the server to collect local model updates from the user's device and produces a weighted aggregation of the models to obtain a new global model. This procedure is repeated until the global model reaches convergence. The FedAvg algorithm is also used in our proposed method.

B. Transfer Learning (TL)

Transfer learning (TL) is a machine learning technique that aims to extract knowledge gained in a source task/domain and use it to facilitate the learning of a target predictive function

in a different task/domain [22], [58]. In recent years, TL research has resulted in effective learning techniques capable of addressing a number of problems in various areas [17], [18]. Using DNNs, TL has gained significant attention among the ML community [22], [58], where a commonly used approach is the TL parameter sharing strategy, especially in network-based approaches.

C. FL and TL

Chen et al. [7] were the first to propose a FL, TL framework for wearable healthcare, referred to as FedHealth, to preserve patients' privacy and security. A global model is trained based on public datasets in the server and sent to the users. After the update process is performed in each user device, the updated local models of the users are shared with the server. The global model is updated by aggregating the weights and biases using the FedAvg algorithm and then sent back to the users. This procedure is repeated with the continuously emerging user data. Zhao et al. [56] proposed a network anomaly detection method (referred to as NAFT) which uses FL and TL to tackle the issue of non-IID data among multiple users. An anomaly detection model was trained using FL on the benign network data on the users' devices. After the global model was aggregated on the server, a TL approach was used to reconstruct and fine-tune the global detection model for a specific anomaly detection task.

III. RELATED WORK

Research in the field of phishing detection can be categorized as follows:

Blacklist/Whitelist Approach. A database which includes a blacklist and whitelist created from reliable resources to determine whether a URL link exists in the blacklist (phishing websites) or whitelist (benign) [23], [6]. This approach determines whether the URL is phishing or not by cross checking it with the URLs listed on the blacklist and whitelist and therefore cannot detect unknown phishing URLs. In addition, it necessitates frequent updating of the lists and utilizes excessive system resources [44].

Website Content-Based Approach. This approach is used to detect phishing websites based on visual similarity (e.g., block-level, layout and style) [53], [34] and textual similarity [54] of web pages' snapshots to the examined website. While this approach can detect successfully phishing websites with only a few false alarms, it is usually accompanied by significant runtime complexity and extensive calculation and storage resources [54].

URL Character-Based Approach. In this approach, the URLs' characters are analyzed and features from the URL are extracted to train machine learning (ML) and DL models which are used to detect whether the URL is phishing or not. Sahingoz et al. [45] explored natural language processing (NLP) techniques to determine whether the URL is phishing or not. Features were selected from the URLs to create an anti-phishing system, and seven different ML classifiers were induced. Bahnsen et al. [3] compared a random forest

classifier that used manual features engineering with a DL method based on recurrent neural networks (RNNs) that used embedding representation of the URL's characters. The authors showed that the DL model provided better results than the ML model (with a 5% improvement in the accuracy results), and more efficient in terms of speed and memory consumption. Wang et al. [49] used a DL network architecture based on a bidirectional LSTM and 1D-CNN. Wang et al. encoded each character in the URL using the word2vec [43] algorithm which served as input to a DL network. The bidirectional LSTM is used to extract the URL's global features, and the CNN is used to automatically decide which characters have an essential meaning for phishing detection, in order to obtain the main key components of the URL and compress the features that are extracted into a fixed-length vector space.

However, in real-life environments, using supervised models to detect phishing URLs is less applicable as users' URL entries are unlabeled. Moreover, the existing methods mentioned above require centralized training data [37] and there is a need to collect URL data from different organizations/users without compromising the users' privacy. To address these issues, we proposed our method, a semi-supervised approach that combines anomaly detection by using FL algorithm [27], [30]. This approach learns the representation of normal data (benign URLs) and identifies outliers in the data (phishing URLs) using a FL algorithm that allows training across multiple decentralized edge devices without sharing data samples. Khramtsova et al. [37] proposed FL approaches for detecting malicious URLs, and Ongun et al. [39] introduce CELEST, a federated learning framework for global threat detection. This framework employs federated learning techniques, utilizing federated FastText to generate feature embeddings and train a neural network to detect malicious URLs. Both of these research studies rely on supervised learning and are unsuitable for real-life scenarios where users' URL entries are unlabeled.

IV. THE PROPOSED METHOD

Design considerations: Table I describes the challenges faced in detecting phishing URLs and the solutions of the STFL method for those challenges.

Training procedure: According to the training procedure, as shown in Fig. 1, we elaborate on each step as follows: **Step (a):** we initialize (on the server) the global Bi-LSTM autoencoder network. An LSTM autoencoder [47] is an implementation of an autoencoder for sequential data in which an LSTM is used in the encoder/decoder architecture. By using a Bi-LSTM in the autoencoder, the autoencoder can capture the lexical dependencies between the characters from left-to-right and right-to-left in the sequential data. To build the Bi-LSTM autoencoder network, we divide the URL into three parts which are represented as one-hot encoding: domain and top level domain (TLD), sub-domain, and the path, as can be seen in Fig. 2(a). Hence, the Bi-LSTM autoencoder network is composed of three Bi-LSTM autoencoder models, where each model receives a different part of the URL as input. The first model receives the domain and TLD of the URL,

the second model receives the sub-domain of the URL, and the third model receives the path of the URL. The goal is to analyze all of the parts in order to obtain as much information as possible from each part. As shown in Fig. 1(a) the server sends the global Bi-LSTM autoencoder to each of the selected users' devices. **Step (b):** According to FL paradigm, each user uses this global Bi-LSTM autoencoder locally (i.e., a local network) to train on the unlabeled URL data from their device; i.e., an unsupervised training process is performed (see Fig. 1(b)). **Step (c) and (d):** The trained local Bi-LSTM autoencoder from each user is sent back to the server (see Fig. 1(c)) and aggregated into a global Bi-LSTM autoencoder network by utilizing the most common optimization methods for FL, the FedAVG algorithm [35] (see Fig. 1(d)). **Step (e):** This process is considered one training round, and multiple training rounds are repeated until the global Bi-LSTM autoencoder network reaches convergence. **Step (f):** Once the global Bi-LSTM autoencoder network has been updated, the encoder can be used to encode sequence data as a feature vector for a supervised learning model [47]. The server takes the encoder from the updated global Bi-LSTM autoencoder network and induces a classification model. The classification model is trained using the learned behavioral patterns from each user, combined with the learned behavioral patterns based on the open publicly available labeled dataset that exists on the server, i.e., in a semi-supervised TL, FL-based approach (see Fig. 1(f)).

Algorithm 1: When presenting the method, the following notation is used: K is the number of the users, k is an index of the participating user, D_{PL} is the public labeled dataset (X, Y) , X are the URL samples, Y are the labels of the URL samples, t is the training round, w^{ag} is the global Bi-LSTM autoencoder network, w_t^{ag} is the global Bi-LSTM autoencoder network in training round t , w_t^{ak} is the local Bi-LSTM autoencoder network according to user k in training round t , w^c is the classification model in the server, U_k are selected participating users i.e., sub group in each training round t , n_k is the size of local training data of user k , N is the size of all users' local training data. First, we initialize the weights of the global Bi-LSTM autoencoder network w_t^{ag} on the server (line 1). In each training round t , the server sends a global Bi-LSTM autoencoder network w_t^{ag} to selected participating users (lines 2-4); each user's device $k \in U_k$ 1) performs training unsupervised learning using the Bi-LSTM autoencoder network w_t^{ag} on their local unlabeled dataset consisting of n_k local training data examples, and 2) updates the local network accordingly w_{t+1}^{ak} (lines 5-7). The server then collects all of the updated local Bi-LSTM autoencoder networks w_{t+1}^{ak} from the selected participating users and produces a weighted aggregation of the networks, using the FedAvg algorithm to update the global Bi-LSTM autoencoder network w_{t+1}^{ag} . This procedure is repeated in multiple training rounds (lines 8-10). After the updated global Bi-LSTM autoencoder network reaches convergence, the server creates a classification model by taking the encoder from the updated global Bi-LSTM autoencoder network w_T^{ag} , and adds

TABLE I Challenges of detecting phishing URLs and the suggested solutions by using STFL method.

Challenge	Description of Challenge	Solved by STFL method
Users' privacy	URL data must be collected from different users (or organizations) without compromising sensitive data.	Our method uses a straightforward FL environment. Only the local model parameters from each user's device are shared to the central server to preserve data privacy.
Unlabeled data	In real-life settings, when entering a website using the users' devices there is no indication if the website is legitimate (benign) or not, i.e., the users' URL entries are unlabeled. Note that existing methods assume that labels are available (supervised learning) even though they are not.	Our method focuses on real-life settings in which the data are unlabeled.
Malicious entries	Most users' entries are benign URLs.	Our method uses anomaly detection to learn the representation of the normal data (benign URLs) and identify the outlying data (phishing URLs) on the user's device.
User's behavior	Inducing a model based on all of the available labeled data does not take into account each user's unique behavioral patterns.	Our method simulates an environment in which we can induce a model according to each user's device in order to learn the representation of that user's unlabeled data.
Combining knowledge	It is necessary to transfer the learned patterns (from each user's device) to the learned patterns from the open publicly available labeled data.	Our method uses transfer learning.
non-IID	In real-life settings, users' data are heterogeneous, i.e., data from each user's device are not representative and is non-identically distributed across all other users (non-IID data).	By conducting unsupervised learning on the user's device, our method is not affected by the non-IID data since training the model on the user's device does not require labels.

Algorithm 1: Semi-supervised transfer-learning, federated-learning approach to detect phishing URL attacks (the proposed STFL phishing URL method)

Input: K - Number of users
 $D_{PL}(X, Y)$ - Public labeled dataset (URL, Label)
 k - Index of the participating user
 U_k - Selected participating users
 t - Training round
 w^{ag} - Global Bi-LSTM autoencoder network
 w^{ak} - Local Bi-LSTM autoencoder network at user's device k
 w^c - Classification model in the server
 n_k - Size of local training data of user k
 N - Size of all users' local training data
Output: w_T^{ag}, w^c

```

1 initialization:  $w_t^{ag}, w_t^c$  at  $t = 0$ 
2 foreach training round  $t = 0, 1, 2, \dots, T$  do
3    $U_k \leftarrow \text{SelectSubGroupUsers}(K)$ 
4   Send  $w_t^{ag}$  to each user  $k \in U_k$ 
5   foreach user  $k \in U_k$  do
6      $w_{t+1}^{ak} \leftarrow \text{TrainLocal}(w_t^{ag})$  ▷ On the user device  $k$ 
7   end
8   Send  $w_{t+1}^{ak}$  from each user  $k \in U_k$  to the server
9    $w_{t+1}^{ag} \leftarrow \sum_{k \in U_k} \frac{n_k}{N} w_{t+1}^{ak}$  ▷ FedAvg on the server
10 end
11  $\bar{C} \leftarrow \text{CreateEncoderClassifier}(w_T^{ag})$  ▷ On the server
12  $w^c \leftarrow \text{TrainClassifierServer}(\bar{C}, D_{PL})$  ▷ On the server

```

fully connected layers to the encoder $\bar{C}$. The classification model is trained using supervised learning on the labeled dataset D_{PL} (line 11). The server performs transfer learning and transfers the learned patterns of the weights and the bias of the user's encoder to the classification model and trains just the fully connected layers in the classification model (line 12). During the entire process, the data does not leave the user's device; only the updated model's parameters are sent between the user's device and the server. In addition, the classification model in the server is trained until the global

Bi-LSTM autoencoder network gets to convergence; however, there is a need to validate the global Bi-LSTM autoencoder network by training the classification model in each training round (will be elaborated in the experimental setting).

V. EVALUATION

A. Datasets

CPURNN dataset [3] contains more than two million URLs: 1,146,432 phishing URLs from PhishTank and 1,200,000 benign URLs from Common Crawl.

PDRCNN dataset [49] contains approximately 500,000 URLs: 245,385 phishing URLs from PhishTank from August 2006 to March 2018, and 245,023 benign URLs that were collected as follows: the authors used search engines to search for the domain names that appear in the Alexa 1M URLs. For each search, the top 10 URLs in the results were collected, and then the URLs that could not be reached were filtered.

Dataset Khramtsova et al. [25] Approximately 700K samples of benign and malicious URLs that were collected from various sources (Hacker News post URLs, URLHaus, OpenPhish, PhishTank). For our experiments, we used 24,002 phishing URLs and 367,384 benign URLs from the Khramtsova et al. dataset. The data are publicly available in GIT¹.

B. Experimental Setting

Pre-Process the Data: each URL was divided into three parts: domain with TLD, subdomain, and the path.

The size of each part was empirically set to 45 according to the average size and standard deviation. Each character from the URL was indexed to a numeric number. The 52 characters with the highest frequency were indexed after exploring all of the characters in the URLs datasets (e.g., English letters, digits, and special symbols such as: @, &, and ?). Then,

¹https://github.com/khramtsova/url_feature_extractor/blob/master/urls_final_complete.tar.xz

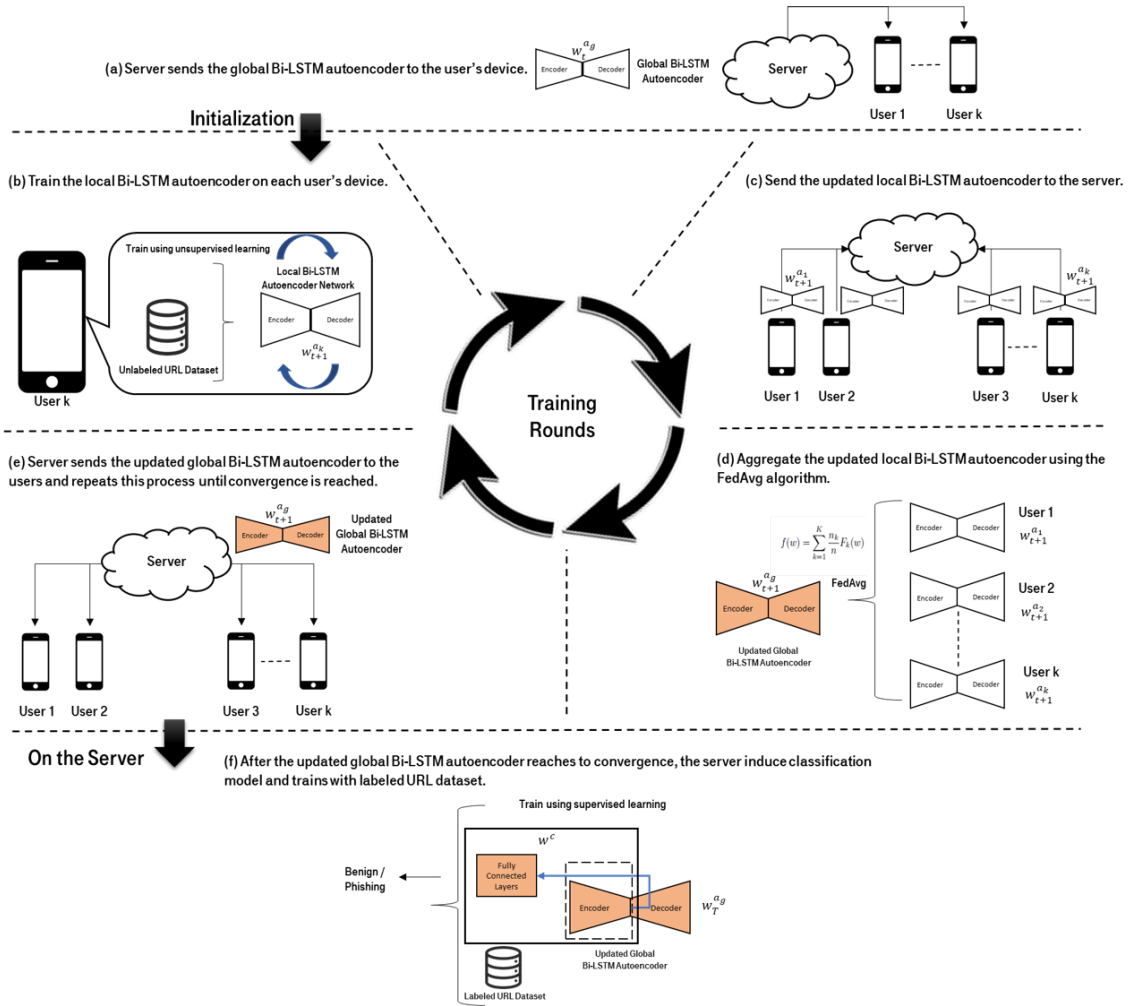


Fig. 1 The training procedure of our STFL detecting phishing URL.

each index character was represented as a vector of one-hot encoding. Consequently, each URL was represented by two dimensions: the length of the URL $\times$ the size of the indexes (45 for each input: domain with TLD, subdomain, and the path $\times$ 52 indexes each).

Structure of the Bi-LSTM Autoencoder Network: The structure of the Bi-LSTM autoencoder network is shown in Fig. 2(a). Each model (domain with TLD, sub-domain, and the path) in the network has the same layers. The size of the input is the size of the URL (represented as a matrix of one-hot encoding), which is input to the Bi-LSTM layer with 64 units in the hidden layer, then a repeated vector with a dense layer with the size of the URL length is added, which is input to Bi-LSTM layer with 64 units in the hidden layer. The final layer is a dense layer of the same size as the input size with the Sigmoid activation function.

Structure of the Bi-LSTM Classification Model: shown in Fig. 2(b), a fully connected layer of 128 dimensions with Relu is added to the encoders of each model in the Bi-LSTM autoencoder network. Then all of the fully connected layers are concatenated together. Two hidden linear layers with ReLU, where the output dimensions are 64 and 32 respectively

composed to the concatenated layer. The final output consists of one neuron with a Sigmoid activation function. The network was trained to minimize binary cross-entropy, several dropouts were added to the network to avoid overfitting and accelerate network training, and Adam [26] optimizer was used as an optimization algorithm. Two experiments were conducted.

First Experiment: We compared our STFL Phishing URL method to the state-of-the-art method proposed by Khramtsova et al. [25], which is based on supervised learning (unrealistic in a real-world scenario). We induced the same supervised learning classification model proposed by Khramtsova et al. [25] in both FL and CL environments based on the code provided by the authors. In addition, we trained our STFL classification model as supervised learning in FL and CL environments for comparison. Finally, we trained our STFL method as semi-supervised learning and compared its results to the results obtained by using supervised learning of both our method and that of Khramtsova et al., as described above.

Considering that the dataset of Khramtsova et al. contains only benign and phishing URLs (and not all malicious URLs). We divided the Khramtsova et al. dataset into a training and testing set (90%:10%). The training set contained 330,645

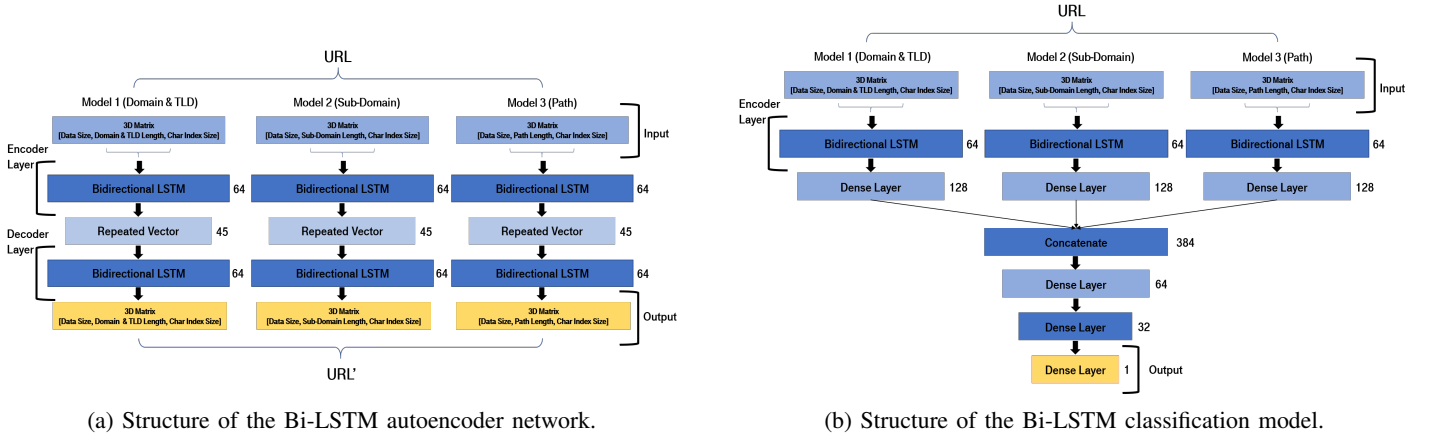
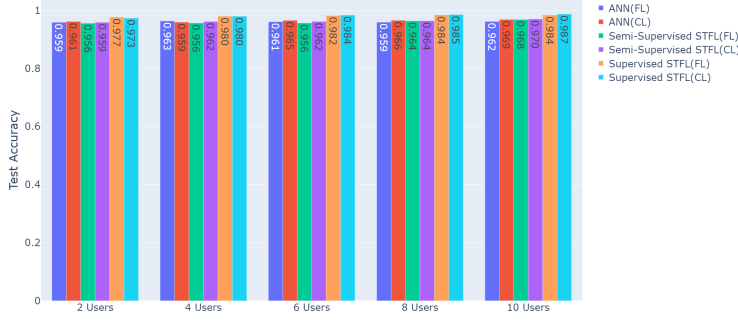


Fig. 2 STFL structures models

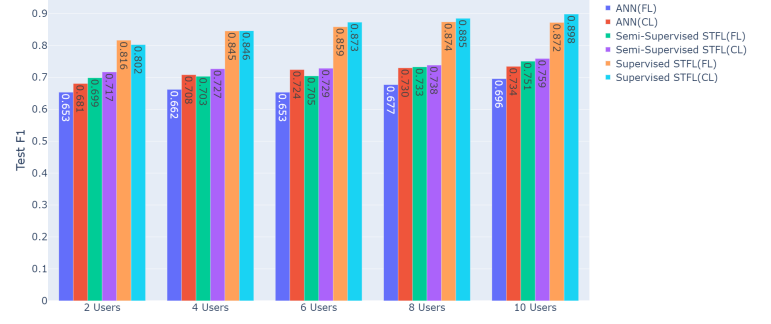
benign and 21,602 phishing URLs, and the testing set contained 36,739 benign and 2,400 phishing URL samples. The training set is highly imbalanced, with only 6.53% of the URLs classified as phishing. To address this and obtain a balanced dataset, we used undersampling, to undersample the majority class (benign URLs) and we removed benign URL samples from the training set. The training set used to train the models in the CL and FL environments contained 89,999 benign and 19,442 phishing URLs. After performing undersampling, the training set was still not fully balanced, however continuing to undersample would remove too much data. On the server, the classifier model for the semi-supervised STFL method was trained on the dataset which contained 10,001 benign and 2,160 phishing URLs. The following simulations were used for the supervised learning approach for both the STFL classification model and the ANN model of Khramtsova et al. We collected the benign and phishing URLs samples from the dataset of Khramtsova et al. and randomly divided them among ten participating users (as commonly done in various research studies [35], [33], [57], [25]). The data were equally distributed between the benign and phishing URLs samples based on the sample label. The server induces an initial global classification model and then transfers it to the devices of all participating users. On each user's device, the local classification model is trained on labeled URL data (benign and phishing URLs). After the classification models are trained, the updates to the local classification models are shared with the server. The server aggregates the weights of each local classification model by using the FedAvg algorithm and updates the global classification model. This procedure is repeated for 100 rounds. To simulate the supervised learning of the CL approach, we use the same data samples used for the FL training and train the same classification model for 100 epochs, without dividing the samples among the ten participating users since, as mentioned, in CL all of the data are located centrally on the server. For the semi-supervised STFL model, the same process described above for the two models (the STFL classification model and the

ANN model of Khramtsova et al.) was performed, with the following exceptions: the benign and phishing URL samples were unlabeled, and the server performs TL and supervised learning by training the classification model using Khramtsova et al. labeled datasets (benign and phishing URLs) which have not been used with the CL and FL training processes. We calculate the accuracy, precision, recall, and F1 score of the test set.

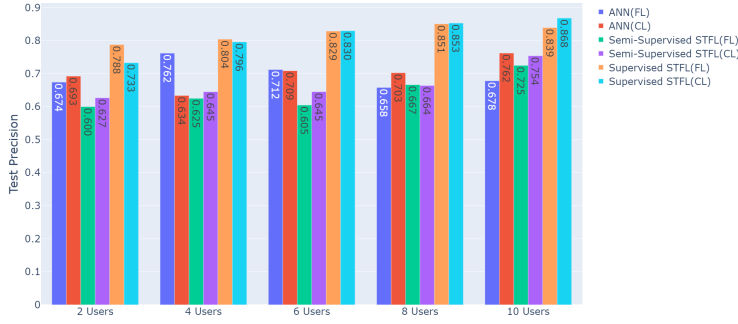
Second Experiment: We compare the FL training process using the STFL phishing URL method to a CL training process. We divided the CPURN [3] dataset into a training and test set (90%:10%). The training set contained 1,080,000 benign URLs which were divided randomly (as commonly done in various research studies [35], [33], [57], [25]) among k participating users, equally distributed between normal (benign URL) samples. For example, if k equals 10, then each user gets 108,000 URLs samples. On the server, the classifier model was trained with the PDRCNN [49] dataset (a different dataset was used for the TL process) that contains 440,921 URL samples (220,512 benign and 220,409 phishing). The test set used to evaluate the methods' performance is from the CPURN [3] dataset and contains 120K benign and 111K phishing URL samples. In order to simulate a real-life setting i.e., an environment in which each user has his/her own data, we used the dataset that contains the most benign URLs, which is the CPURN [3] dataset. Then the process described in Algorithm 1 is performed (FL). The server performs TL and supervised learning by training the classification model using a different dataset, the PDRCNN [49], which is labeled and contains benign and phishing URLs. Note that the classification model trains only the fully connected layers. This procedure is repeated in 100 training rounds. To simulate the CL training process, the same normal (benign URLs) samples from the CPURN [3] dataset were used without dividing them among k users, since in CL all of the data are located centrally on the server. In each epoch, the Bi-LSTM autoencoder network is saved and a classifier model is induced by taking the encoder and adding fully connected layers to it.



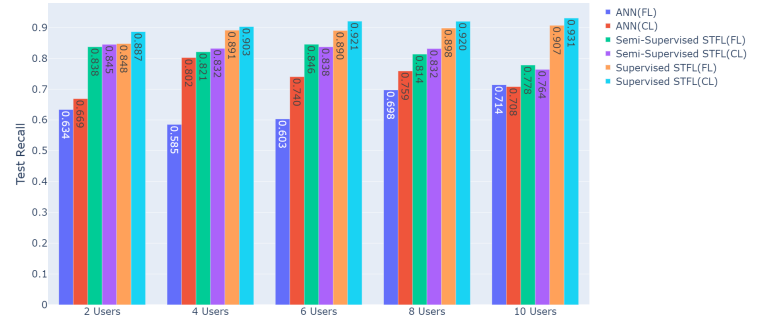
(a) Test accuracy results as a function of the number of users.



(b) Test F1 results as a function of the number of users.



(c) Test precision results as a function of the number of users.



(d) Test recall results as a function of the number of users.

Fig. 3 Final results for the measurements accuracy, precision, recall, and F1 score on the test set for each model: supervised STFL, semi-supervised STFL, and ANN (Khramtsova et al. [25] model), on both the CL and FL environments.

Inference Time: In these experiments, we evaluated the performance of the classification model on the server with the test set every five training rounds. After the classification model is trained on five training rounds, the server uses the classification model to predict the test set samples' class and compares the probability predictions with the true labels of the test set samples. In the first experiment, we randomly divided the dataset among ten participating users, and, then we selected $k = \{2, 4, 6, 8, 10\}$ participating users and compared the results of the $k = \{2, 4, 6, 8, 10\}$ participating users. In the second experiment, we randomly divided the data set among $k = \{10, 50\}$ participating users. For each of the k users (in both experiments), three-fold was performed, and the results were averaged to remove noise which might result from the random decisions made by the DL algorithms and the random data split.

Please note that in all of our experiments we assume that none of the participating users are adversarial users. In the second experiment, all of the users participated in the FL training process.

C. Experimental Results

1) **Experiment 1:** is designed to answer the following research questions:

- Does the supervised STFL classification model and supervised ANN model of Khramtsova et al. outperform

the semi-supervised STFL model in both the FL and CL environments?

- Does the STFL model achieve higher performance results in CL and FL environments than the state-of-the-art method (i.e., Khramtsova et al.)?
- What effect does the $k = \{2, 4, 6, 8, 10\}$ participating users have on the preformance of the models?

Khramtsova et al. dataset are used in the first experiment, as explained in the experimental setting. The results of the first experiment are presented in Fig. 3, which includes four sub-graphs that present the final accuracy, precision, recall, and F1 score results on the test set for each model: supervised STFL, semi-supervised STFL, and ANN (the Khramtsova et al. model) in both the CL and FL environments. For each sub-graph, we present the final results of the models as a bar graph for each $k = \{2, 4, 6, 8, 10\}$ participating users. Sub-graph (a) presents the accuracy results; as can be seen, the accuracy score for each $k = \{2, 4, 6, 8, 10\}$ participating users is high for all of the models. In addition, we observe that the more users there are, the higher the accuracy, for all the models and in both environments; moreover, there is no noticeable difference in the models' results between the CL and FL environments. Since accuracy is not a suitable metric for unbalanced data, we also measure the precision, recall, and F1 score. Sub-graphs (c) precision, (d) recall, and (b) F1 score present the results for each model in both environments.

The F1 score [16] combines the precision and recall of the performance model, and it is defined as the harmonic mean of the models' precision and recall. Therefore, we refer to the F1 score metric when analyzing the results.

The ANN model's results are presented in dark blue and red bar graphs for the CL and FL environments respectively in the sub-graphs for each $k = \{2, 4, 6, 8, 10\}$ participating users. As shown in the sub-graphs, the ANN model in both environments is stable and the F1 score improves when there are more users. However, the ANN model obtains low F1 score values against the semi-supervised and supervised STFL models for each $k = \{2, 4, 6, 8, 10\}$. The semi-supervised STFL model's results are presented in green and purple bar graphs on the CL and FL environments respectively in the sub-graphs for each $k = \{2, 4, 6, 8, 10\}$ participating users. As shown in the sub-graphs, the semi-supervised STFL model's results are stable and better than those of the ANN model in both the CL and FL environments for each $k = \{2, 4, 6, 8, 10\}$. As seen in sub-graph (b) for the F1 score, the score improves when there are more users. When there are ten users, the model obtains an F1 score of approximately 75% in both CL and FL environments. Moreover, when there are 8 and 10 participating users, it is noticeable that even the semi-supervised model in the FL environment obtains higher results from the ANN model in both environments. The supervised STFL model's results are presented in orange and light blue bar graphs for the CL and FL environments respectively in the sub-graphs for each $k = \{2, 4, 6, 8, 10\}$ participating users. As seen in the sub-graphs, the supervised STFL model's results are stable and it obtains better results in both environments than the semi-supervised STFL model and ANN model for each $k = \{2, 4, 6, 8, 10\}$. Similar to the semi-supervised setup, as shown in sub-graph (b) for the F1 score, there is improvement in the score when there are more users. When there are ten users, the model obtains an F1 score of approximately 87% in both environments. In addition, the F1 score of the supervised STFL in the CL environment was 3% higher than the F1 score of the supervised STFL in the FL environment.

2) **Experiment 2:** is designed to answer the following research question: Can a semi-supervised TL, FL approach be used for the detection of phishing URL attacks and perform as well as the CL approach?

CPURNN [3] and PDRCNN [49] datasets are used in the second experiment, as explained in the experimental setting. Fig. 4(a) and Fig. 4(b) present the accuracy and loss function results of the training and testing performance for the CL and FL environments on the public dataset for $k = \{10, 50\}$ users. In the training and test sets, the accuracy and loss function results of the FL and CL obtained the same convergence results. However, we can see that the FL presented results are more stable than the CL results. In addition, even though each user receives very small number of samples (21,600 benign URLs) when the data are divided among 50 users, the STFL method still achieves high results in both the CL and FL environments.

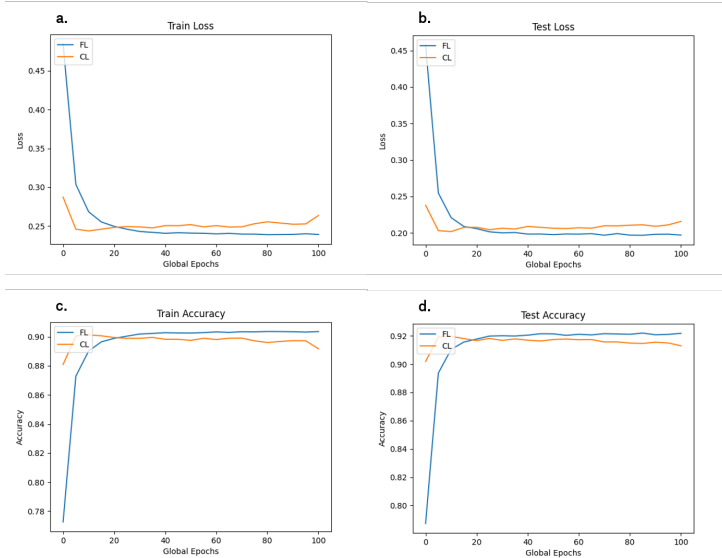
D. Discussion

Security Risks. In this paper, we presented a novel approach to detect phishing URL attacks using semi-supervised, transfer-learning, and federated-learning which is applicable to real-life scenarios where the user's URLs entries are unlabeled. Similar to previous state-of-the-art research, which combined FL to detect malicious URLs (among them phishing) [25], we assumed that all participating users in the FL process are non-malicious users. However, a possible threat model scenario is to be considered in which adversaries could manipulate the parameters of the global model by manipulating the local models' parameters during the updating process. Our proposed method can handle this scenario as follows: (1) Improving the robustness against label-flipping attacks does not require any enhancements since our local model is an autoencoder that relies on the URL characters and does not rely on the URL labels. (2) In the case of backdoor attacks, where the adversary changes the patterns of the URLs in the user's device, it is possible to use publicly available solutions [24], [21]. For example, Khazbak et al. [24] proposed MLGuard which computes cosine similarity scores between measured angular distances of the parameter updates (of users' local models) and excludes those with the lowest similarity scores. (3) Our method can be enhanced by adding an additional pre-trained global classification model, whose task is to check the URLs entries and filter the benign URLs from the data. The rest of our method remains unchanged since the process continues to work on already-filtered data.

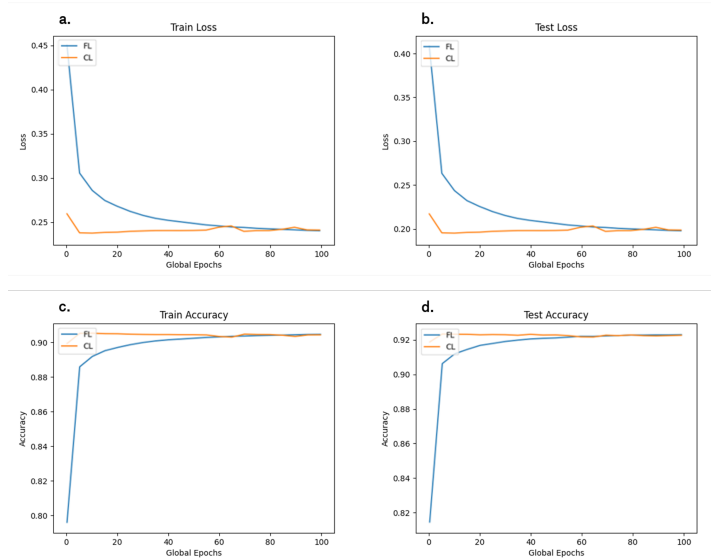
Additional threats that need to be considered are privacy threats that might risk users' data. Those threats in FL can be divided into the following general categories: 1) Insider adversary — honest-but-curious and malicious users participating in the learning rounds [48], [37]. 2) Outsider adversary — eavesdroppers on the channel can appear when communicating the updates between trusted users and server [13]. 3) Malicious server — an honest-but-curious server can inspect users' updates without altering the model, and a potentially malicious server can inspect the updates and also can tamper with the model [51]. Several possible solutions have been proposed for secure privacy-preserving FL systems for each threat: 1) Differential Privacy Stochastic Gradient Descent (DP-SGD), learns general patterns and not details about specific training examples, which ensures that no individual examples would be memorized by the model [1], [52]. 2) Secure Aggregation, which is a method that secretly sums up all the updated models, so that in the server only the aggregated result can be seen. Since the result comes from all user participants, the server can learn hardly anything meaningful about a single user participant [5]. 3) Homomorphic Encryption (HE), is a form of encryption that protects users' data by performing computation directly on ciphertexts [29].

Implementation Notes.

The implementation of the data aggregation in the proposed method was done via pySpark, i.e., having a corresponding client (Python 3.x) and Spark 2.3.0 (or later) on the Cloudera



(a) Using 10 users in the second experiment.



(b) Using 50 users in the second experiment.

Fig. 4 Compare the training and testing results between CL and FL.

Hadoop 5.3 cluster, and the proposed method algorithm was tested using two Nvidia Geforce 2080 TI GPU cards, CPU intel core i7-9700k @ 3.00ghz and 64GB DDR4 Memory. The proposed method was tested on unsupervised data collected for two weeks from a European telecommunications company (the data and the results can not be shared due to privacy reasons).

VI. CONCLUSION AND FUTURE WORK

We presented a semi-supervised transfer-learning federated-learning approach for the detection of phishing URL attacks, which combines both unsupervised learning (for learning users' URLs patterns) and classification model (for classifying URLs). Our experiments showed that STFL successfully obtained the same results as in the CL environment and similar results to using supervised learning in the FL environment. However, STFL is suitable for real-life settings in which users' URL entries are unlabeled and heterogeneous; it also considers each user's unique behavioral patterns, assuming that most users enter benign URLs. Moreover, STFL does not compromise users' sensitive data and warns users if a phishing website is detected. Future work may include improving our method by aggregating the models with the lowest loss function score and considering adversarial users involvement in the FL environment. Furthermore, phishing URLs have an explicit temporal nature. Therefore, it should use the current URLs data to train the model and detect future URLs. The datasets we used were gathered over a long period. Thus, there is a need to collect a dataset from users' entries partitioned by their temporal orders.

REFERENCES

- [1] Abadi, M., Chu, A., Goodfellow, I., McMahan, H.B., Mironov, I., Talwar, K., Zhang, L.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. pp. 308–318 (2016)
- [2] (APWG), G.A.: Phishing activity trends report. <https://apwg.org/trendsreports/> (4th Quarter 2023)
- [3] Bahnsen, A.C., Bohorquez, E.C., Villegas, S., Vargas, J., González, F.A.: Classifying phishing urls using recurrent neural networks. In: 2017 APWG symposium on electronic crime research (eCrime). pp. 1–8. IEEE (2017)
- [4] Birlea, M.C.: Phishing attacks: Detection and prevention. arXiv preprint arXiv:2004.01556 (2020)
- [5] Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1175–1191 (2017)
- [6] Browsing, G.S.: Google developers accessed. <https://developers.google.com/safe-browsing> (July 24, 2019)
- [7] Chen, Y., Qin, X., Wang, J., Yu, C., Gao, W.: Fedhealth: A federated transfer learning framework for wearable healthcare. IEEE Intelligent Systems **35**(4), 83–93 (2020)
- [8] Chiew, K.L., Yong, K.S.C., Tan, C.L.: A survey of phishing attacks: Their types, vectors and technical approaches. Expert Systems with Applications **106**, 1–20 (2018)
- [9] Dai, X., Spasić, I., Meyer, B., Chapman, S., Andres, F.: Machine learning on mobile: An on-device inference app for skin cancer detection. In: 2019 Fourth International Conference on Fog and Mobile Edge Computing (FMEC). pp. 301–305. IEEE (2019)
- [10] Doriguzzi-Corin, R., Siracusa, D.: Flad: adaptive federated learning for ddos attack detection. Computers & Security **137**, 103597 (2024)
- [11] Douzi, S., Amar, M., El Ouahidi, B.: Advanced phishing filter using autoencoder and denoising autoencoder. In: Proceedings of the International Conference on Big Data and Internet of Thing. pp. 125–129 (2017)
- [12] Feng, J., Zou, L., Nan, T.: A phishing webpage detection method based on stacked autoencoder and correlation coefficients. Journal of computing and information technology **27**(2), 41–54 (2019)
- [13] Fredrikson, M., Jha, S., Ristenpart, T.: Model inversion attacks that exploit confidence information and basic countermeasures. In: Proceedings of the 22nd ACM SIGSAC conference on computer and communications security. pp. 1322–1333 (2015)
- [14] Ghosh, A., Senthilrajan, A.: An approach for detecting spear phishing using deep packet inspection and deep flow inspection. In: Proceedings

- of the 5th International Conference on Cyber Security & Privacy in Communication Networks (ICCS) (2019)
- [15] Graves, A., Schmidhuber, J.: Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural networks* **18**(5-6), 602–610 (2005)
 - [16] Grishman, R., Sundheim, B.M.: Message understanding conference-6: A brief history. In: *COLING 1996 Volume 1: The 16th International Conference on Computational Linguistics* (1996)
 - [17] Grolman, E., Bar, A., Shapira, B., Rokach, L., Dayan, A.: Utilizing transfer learning for in-domain collaborative filtering. *Knowledge-Based Systems* **107**, 70–82 (2016)
 - [18] Grolman, E., Finkelshtein, A., Puzis, R., Shabtai, A., Celniker, G., Katzir, Z., Rosenfeld, L.: Transfer learning for user action identification in mobile apps via encrypted traffic analysis. *IEEE Intelligent Systems* **33**(2), 40–53 (2018)
 - [19] Gupta, S., Singhal, A., Kapoor, A.: A literature survey on social engineering attacks: Phishing attack. In: *2016 international conference on computing, communication and automation (ICCCA)*. pp. 537–540. IEEE (2016)
 - [20] Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., Eichner, H., Kiddon, C., Ramage, D.: Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604* (2018)
 - [21] He, L., Karimireddy, S.P., Jaggi, M.: Secure byzantine-robust machine learning. *arXiv preprint arXiv:2006.04747* (2020)
 - [22] Iman, M., Arabnia, H.R., Rasheed, K.: A review of deep transfer learning and recent advancements. *Technologies* **11**(2), 40 (2023)
 - [23] Jain, A.K., Gupta, B.B.: A novel approach to protect against phishing attacks at client side using auto-updated white-list. *EURASIP Journal on Information Security* **2016**(1), 9 (2016)
 - [24] Khazbak, Y., Tan, T., Cao, G.: Mlgaurd: Mitigating poisoning attacks in privacy preserving distributed collaborative learning. In: *2020 29th International Conference on Computer Communications and Networks (ICCCN)*. pp. 1–9. IEEE (2020)
 - [25] Khrantsova, E., Hammerschmidt, C., Lagrass, S., et al.: Federated learning for cyber security: Soc collaboration for malicious url detection. In: *IEEE International Conference on Distributed Computing Systems (ICDCS)* (2020)
 - [26] Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014)
 - [27] Konečný, J., McMahan, B., Ramage, D.: Federated optimization: Distributed optimization beyond the datacenter. *arXiv preprint arXiv:1511.03575* (2015)
 - [28] Korkmaz, M., Sahingoz, O.K., Diri, B.: Feature selections for the classification of webpages to detect phishing attacks: A survey. In: *2020 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*. pp. 1–9. IEEE (2020)
 - [29] Li, J., Kuang, X., Lin, S., Ma, X., Tang, Y.: Privacy preservation for machine learning training and classification based on homomorphic encryption schemes. *Information Sciences* **526**, 166–179 (2020)
 - [30] Li, T., Sahu, A.K., Talwalkar, A., Smith, V.: Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* **37**(3), 50–60 (2020)
 - [31] Li, W., Milletari, F., Xu, D., Rieke, N., Hancox, J., Zhu, W., Baust, M., Cheng, Y., Ourselin, S., Cardoso, M.J., et al.: Privacy-preserving federated brain tumour segmentation. In: *International Workshop on Machine Learning in Medical Imaging*. pp. 133–141. Springer (2019)
 - [32] Lu, S., Yao, Y., Shi, W.: Collaborative learning on the edges: A case study on connected vehicles. In: *2nd {USENIX} Workshop on Hot Topics in Edge Computing (HotEdge 19)* (2019)
 - [33] Luping, W., Wei, W., Bo, L.: Cmf: Mitigating communication overhead for federated learning. In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. pp. 954–964. IEEE (2019)
 - [34] Mao, J., Li, P., Li, K., Wei, T., Liang, Z.: Baitalarm: detecting phishing sites using similarity in fundamental visual features. In: *2013 5th International Conference on Intelligent Networking and Collaborative Systems*. pp. 790–795. IEEE (2013)
 - [35] McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: *Artificial Intelligence and Statistics*. pp. 1273–1282. PMLR (2017)
 - [36] Muhammad, K., Wang, Q., O'Reilly-Morgan, D., Tragos, E., Smyth, B., Hurley, N., Geraci, J., Lawlor, A.: Fedfast: Going beyond average for faster training of federated recommender systems. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. pp. 1234–1242 (2020)
 - [37] Nasr, M., Shokri, R., Houmansadr, A.: Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In: *2019 IEEE Symposium on Security and Privacy (SP)*. pp. 739–753. IEEE (2019)
 - [38] Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.R.: Dfot: A federated self-learning anomaly detection system for iot. In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. pp. 756–767. IEEE (2019)
 - [39] Ongun, T., Boboila, S., Oprea, A., Eliassi-Rad, T., Hiser, J., Davidson, J.: Celest: Federated learning for globally coordinated threat detection. *arXiv preprint arXiv:2205.11459* (2022)
 - [40] Preuveneers, D., Rimmer, V., Tsingenopoulos, I., Spooren, J., Joosen, W., Ilie-Zudor, E.: Chained anomaly detection models for federated learning: An intrusion detection case study. *Applied Sciences* **8**(12), 2663 (2018)
 - [41] Ramaswamy, S., Mathews, R., Rao, K., Beaufays, F.: Federated learning for emoji prediction in a mobile keyboard. *arXiv preprint arXiv:1906.04329* (2019)
 - [42] Rao, R.S., Vaishnavi, T., Pais, A.R.: Phishdump: A multi-model ensemble based technique for the detection of phishing sites in mobile devices. *Pervasive and Mobile Computing* **60**, 101084 (2019)
 - [43] Rong, X.: word2vec parameter learning explained. *arXiv preprint arXiv:1411.2738* (2014)
 - [44] Safi, A., Singh, S.: A systematic literature review on phishing website detection techniques. *Journal of King Saud University-Computer and Information Sciences* **35**(2), 590–611 (2023)
 - [45] Sahingoz, O.K., Buber, E., Demir, O., Diri, B.: Machine learning based phishing detection from urls. *Expert Systems with Applications* **117**, 345–357 (2019)
 - [46] Saxe, J., Berlin, K.: expose: A character-level convolutional neural network with embeddings for detecting malicious urls, file paths and registry keys. *arXiv preprint arXiv:1702.08568* (2017)
 - [47] Srivastava, N., Mansimov, E., Salakhudinov, R.: Unsupervised learning of video representations using lstms. In: *International conference on machine learning*. pp. 843–852. PMLR (2015)
 - [48] Truex, S., Liu, L., Gursay, M.E., Yu, L., Wei, W.: Demystifying membership inference attacks in machine learning as a service. *IEEE Transactions on Services Computing* (2019)
 - [49] Wang, W., Zhang, F., Luo, X., Zhang, S.: Pdcnn: Precise phishing detection with recurrent convolutional neural networks. *Security and Communication Networks* **2019** (2019)
 - [50] Wang, W., Li, X., Qiu, X., Zhang, X., Brusica, V., Zhao, J.: A privacy preserving framework for federated learning in smart healthcare systems. *Information Processing & Management* **60**(1), 103167 (2023)
 - [51] Wang, Z., Song, M., Zhang, Z., Song, Y., Wang, Q., Qi, H.: Beyond inferring class representatives: User-level privacy leakage from federated learning. In: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. pp. 2512–2520. IEEE (2019)
 - [52] Weng, S., Zhang, L., Feng, D., Feng, C., Wang, R., Klaine, P.V., Imran, M.A.: Privacy-preserving federated learning based on differential privacy and momentum gradient descent. In: *2022 International Joint Conference on Neural Networks (IJCNN)*. pp. 1–6. IEEE (2022)
 - [53] Wenyin, L., Huang, G., Xiaoyue, L., Min, Z., Deng, X.: Detection of phishing webpages based on visual similarity. In: *Special interest tracks and posters of the 14th international conference on World Wide Web*. pp. 1060–1061 (2005)
 - [54] Zhang, H., Liu, G., Chow, T.W., Liu, W.: Textual and visual content-based anti-phishing: a bayesian approach. *IEEE transactions on neural networks* **22**(10), 1532–1546 (2011)
 - [55] Zhang, L., Lei, X., Shi, Y., Huang, H., Chen, C.: Federated learning for iot devices with domain generalization. *IEEE Internet of Things Journal* (2023)
 - [56] Zhao, Y., Chen, J., Guo, Q., Teng, J., Wu, D.: Network anomaly detection using federated learning and transfer learning. In: *International Conference on Security and Privacy in Digital Economy*. pp. 219–231. Springer (2020)
 - [57] Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., Chandra, V.: Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582* (2018)
 - [58] Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., He, Q.: A comprehensive survey on transfer learning. *Proceedings of the IEEE* **109**(1), 43–76 (2020)